

A MINI PROJECT REPORT ON

**CLOUDVAULT: A SECURE MERN CLOUD STORAGE SYSTEM
WITH DIRECT AWS S3 OBJECT ACCESS AND VERSION
CONTROL**

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Submitted by:

BISHETTI HARIDEVAN (Roll No: 23EG112A06)
SURAJ TAKKALLAPALLY (Roll No: 23EG112A54)

Under the Guidance of:

Internal Project Supervisor

Department of Information Technology

DEPARTMENT OF INFORMATION TECHNOLOGY

ANURAG UNIVERSITY

Academic Year: 2025 - 2026

DECLARATION

We, the undersigned, hereby declare that the mini-project report titled "CLOUDVAULT: A SECURE MERN CLOUD STORAGE SYSTEM WITH DIRECT AWS S3 OBJECT ACCESS AND VERSION CONTROL", submitted in partial fulfillment of the requirements for the award of B.Tech Degree in Information Technology, is a record of our own research work carried out under the supervision of the internal faculty guide.

The results embodied in this report have not been submitted to any other University or Institute for the award of any other degree or diploma.

Student Signatures:

1. _____ (BISHETTI HARIDEVAN, 23EG112A06)

2. _____ (SURAJ TAKKALLAPELLY, 23EG112A54)

CERTIFICATE

This is to certify that the mini-project report titled "CLOUDVAULT: A SECURE MERN CLOUD STORAGE SYSTEM WITH DIRECT AWS S3 OBJECT ACCESS AND VERSION CONTROL" is a bonafide work carried out by BISHETTI HARIDEVAN (Roll No: 23EG112A06) and SURAJ TAKKALLAPELLY (Roll No: 23EG112A54) in partial fulfillment of the academic requirements of B.Tech in Information Technology at Anurag University, and is approved for submission.

Internal Guide

Department of IT

Head of Department (IT)

Anurag University

ABSTRACT

CloudVault is a secure, high-fidelity cloud storage and folder collaboration application built on the MERN stack (MongoDB, Express, React, Node.js). Traditional cloud storage models often struggle with high server bandwidth costs and processing bottlenecks during upload and download transfers. To resolve this, CloudVault implements a direct-to-S3 presigned object streaming architecture, where file binary payloads transfer securely between the user browser and AWS S3 storage buckets, bypassing backend server memory altogether.

The platform integrates several advanced features matching enterprise-grade providers. It features direct user-level access controls, folder structures, and dynamic search filter listings. Furthermore, it incorporates automatic File Versioning, allowing users to restore previous versions or purge history. Bulk operations permit batch movements, soft trashing, permanent deletion, and dynamic on-the-fly zip packaging. Sharing flows support unregistered emails (storing pending links until registration) and direct social shortcuts (Telegram, Facebook, WhatsApp) pointing to a stateless, token-authorized public shared preview landing page. Testing logs verify a 100% E2E integration test pass rate on the live HTTPS EC2 deployment.

1. INTRODUCTION & OBJECTIVES

1.1 System Overview

With the exponential rise in daily data generation, secure and lightweight document vaults have become a primary utility for personal and collaborative productivity. Standard implementations route all uploads and download transfers through backend servers. When multiple users upload heavy documents concurrently, it bottlenecks server network channels, spikes CPU/memory utilization, and leads to high infrastructure costs.

CloudVault addresses this structural issue by separating the control plane (metadata registry via MongoDB and Node.js) from the data plane (binary storage via AWS S3 object storage). The client requests a presigned secure signature from the Express server, and then puts the file directly to the S3 bucket. This ensures the Node.js server stays highly responsive, lightweight, and capable of scaling to thousands of concurrent users with minimal compute hardware.

1.2 Key Objectives

- **Direct-to-S3 Presigning:** Implement presigned upload pipelines to streamline browser-to-S3 direct storage.
- **File Versioning:** Deploy automatic file version tracking with instant rollback recovery triggers.
- **Bulk Operations:** Support multi-file select tools for bulk moving, trashing, and live ZIP downloads.
- **Collaborative Sharing:** Expose secure direct share portals to unregistered users using JWT emailTokens.
- **Responsive Design:** Ensure responsive layouts compatible with touch-screen viewports and desktop webviews.

2. SYSTEM REQUIREMENTS SPECIFICATION (SRS)

2.1 Software Requirements

- Operating System: Ubuntu 22.04 LTS (Production Host), macOS / Windows (Development)
- Runtime Environment: Node.js (v20+ / v22+ recommended)
- Frameworks: Express.js (Backend), React.js v19 (Frontend via Vite)
- Database: MongoDB Atlas (M0 Shared instance)
- Storage: AWS S3 Bucket with CORS policy configured
- Server Engine: Nginx reverse web proxy
- Process Daemon: PM2 manager
- SSL Certificates: Let's Encrypt TLS (Certbot mapping)

2.2 Hardware Requirements

- Development Machine: Minimum 8 GB RAM, 2.0 GHz Multi-Core Processor, 50 GB Free Disk Space.
- Production Host: AWS EC2 t2.micro or t3.micro instance (1 vCPU, 1 GB RAM, 8 GB SSD Storage).

3. SYSTEM DESIGN & ARCHITECTURE

3.1 Core Architecture Logic

The system runs a fully decoupled architectural pattern. When the client attempts to upload, it communicates with the Express backend to obtain a secure signature. The file is uploaded directly to AWS S3. Upon completion, the client returns a metadata payload (filename, size, type, S3 key) which is saved in MongoDB. Downloads follow a similar presigning approach, generating inline or attachment URLs.

3.2 Schema Structure (Mongoose Models)

- User Schema: Stores user credentials, hashed passwords (bcrypt), and display themes.
- Folder Schema: Manages directory structures, ownership, and collaborative sharedWith access lists.
- File Schema: Records active file metadata, size, starred states, soft-deleted state, folder locations, sharedWith collaboration arrays, and a nested versions array tracking S3 historical key references.

3.3 Core REST APIs

- GET /api/files/upload-url - Requests presigned S3 PUT signatures.
- POST /api/files/save-metadata - Saves file record after S3 upload.
- GET /api/files/shared-preview/:fileId - Fetches public info using emailToken.
- GET /api/files/download/:s3Key - Direct stream proxy for downloads.
- POST /api/files/bulk-trash / bulk-restore / bulk-delete - Batch operations.

4. TECHNICAL IMPLEMENTATION DETAILS

4.1 S3 Presigning Execution

Backend uses the AWS SDK v3 `@aws-sdk/s3-request-presigner` to construct secure URLs. Once signed, the client React app initiates a standard Axios PUT request containing the file binary. This bypasses the Express server completely, preventing memory exhaustion on the Node runtime.

4.2 Automatic Version Control

If the backend intercepts a `saveMetadata` call with a duplicate filename in the same folder, it renames the existing database record and pushes its old S3 key reference into the document's version history drawer. Restoring a version pulls the selected record from the version array, pushes the current active file into the version history, and switches the main reference instantly.

4.3 Collaborative Sharing and Auto-Linking

Collaborators are added via email. If the email has no account, the record saves with `user: null`. When that email signs up, the `registerUser` controller triggers a database `updateMany` hook, scanning for any shared records matching the email and updating `sharedWith.user` to the new user's `_id`, granting instant dashboard visibility.

4.4 Shared-Preview Route & emailToken

When a document is shared directly via email, the owner generates a signed JWT containing the file ID and the recipient's email. Unregistered users visit the public frontend URL containing the token. The page fetches file info from the public shared-preview route, and presents inline viewers (text, image, pdf, media players) and a download button anonymously.

5. TESTING, VERIFICATION & CONCLUSION

5.1 Automated Testing Results

The application runs automated integration tests (test-advanced-features.js and test-shared-preview.js) validating authorization, versioning draws, collaborative feeds, filters, bulk compression archives, and token-based direct preview page accesses. The tests run successfully on the live HTTPS production endpoint with 100% correctness.

5.2 Conclusion

CloudVault is a complete, scalable, and responsive cloud storage solution. By offloading data transfers directly to S3 and using MongoDB Atlas for metadata registry, the platform achieves excellent performance on standard Free Tier hardware. The project meets all design goals, successfully enabling anonymous shared file views, collaborative sharing, version histories, and bulk operations.